

GCP Packer Debian Linux

Workflow & Architecture Documentation

Generated: June 29, 2026
Version 0.0.1 | Branch: feature/debian12-gcp

PURPOSE Build hardened Debian 12 GCP images using HashiCorp Packer + Ansible automation	PLATFORM Google Cloud Platform — Project: odev-eiaas-dev-vmi-archv-b9ba
CI/CD Jenkins Pipeline — 4-stage: Init → Validate → Build → Deploy	TESTING Molecule test framework with live GCP instances

Table of Contents

1	Project Overview	3
2	Repository Structure	3
3	Configuration & Environment Variables	4
4	Packer Configuration	5
4.1	Variables (variables.pkr.hcl)	5
4.2	Source Definition (gcp-sources.pkr.hcl)	5
4.3	Build Definition (gcp-builds.pkr.hcl)	6
4.4	OS Variables (os-debian.pkvars.hcl)	6
5	Ansible Automation	7
5.1	Playbook Hierarchy	7
5.2	Task Files	7
5.3	Inventory & Group Variables	8
5.4	Ansible Requirements	8
6	Jenkins CI/CD Pipeline	9
7	Molecule Testing Framework	10
8	Python Task Runner (tasks.py)	11
9	Shell Scripts (bin/)	12
10	End-to-End Workflow	13
11	Key Design Decisions	14
12	File Reference Index	15

1. Project Overview

This repository automates the creation of hardened Debian 12 (Bookworm) machine images on Google Cloud Platform using HashiCorp Packer as the image builder and Ansible as the configuration management layer. The resulting images serve as the foundation for production workloads, with CIS-hardening applied before the image is published.

The project targets the GCP project **odev-eiaas-dev-vmi-archv-b9ba**, uses an internal Pulp mirror for package management, and relies on Jenkins for CI/CD orchestration. Molecule provides post-build integration testing by spinning up real GCP instances and verifying Ansible idempotency.

Source image	Pre-hardened Debian 12 GCP image (debian12-hardened-20260626-22) from the source project
Output image	New GCP image: debian12-hardened-YYYYMMDD-{build_number} in family debian12-hardened
Customization	Ansible configures GCP boot agents, APT repositories, and cleans the image
Export	Image exported as RAW tar.gz to GCS bucket for long-term archival
Testing	Molecule spins live GCP instances to verify the resulting image

2. Repository Structure

The repository is organized into the following top-level directories and files:

gcp-builds.pkr.hcl	Main Packer build definition — plugins, locals, build block, provisioners, post-processors
gcp-sources.pkr.hcl	Packer source block: GCP project, network, machine type, image naming
variables.pkr.hcl	All Packer input variables with env() defaults
os-debian.pkvars.hcl	Debian 12 specific variable overrides (post list)
Jenkinsfile	4-stage Jenkins pipeline: Init, Validation, Build, Deploy
tasks.py	PyInvoke task runner — build orchestration, molecule test runner, GCP image purge
requirements.txt	Python dependencies (invoke, ansible, molecule, flake8, etc.)
.env	Environment variable definitions for vCenter and GCP credentials
artifactory-images.json	Image catalog: Artifactory repo paths and Ansible group assignments
ansible/playbooks/	Ansible playbooks — build.yml entry point → gcp_build.yml
ansible/playbooks/tasks/	Reusable Ansible task files: apt setup, GCP boot config, apt cleanup
ansible/inventories/group_vars/	Variable files: all/main.yml (server URLs), debian12.yml (APT repos)
ansible/requirements.yml	Ansible Galaxy requirements: ansible.posix, community.general, google.cloud
ansible/extensions/molecule/	Molecule test suite: create, converge, verify, destroy playbooks
bin/	Shell scripts: init-env.sh, invoke.sh, deploy_gcp.sh, deploy-lab.sh
docs/	Pipeline step definitions and TODO list

pkrrvars-dump/	Example/reference Packer variable dump files
----------------	--

3. Configuration & Environment Variables

The project uses two environment configuration files. The `.env` file provides shell-level variable exports used by build scripts and Packer. Sensitive credentials are injected at runtime by Jenkins using credential bindings.

3.1 Core GCP Environment Variables

GCP_PROJECT_ID	GCP project: odev-eiaas-dev-vmi-archv-b9ba
GCP_CREDENTIALS_FILE	Path to GCP service account JSON keyfile (.gca, injected by Jenkins)
GCP_ZONE	Deployment zone: us-central1-a
GCS_BUCKET	GCS bucket for exported RAW images (injected by Jenkins)
PACKER_SSH_PUBLIC_KEY	SSH public key injected into GCP instance metadata for Packer SSH access
GCP_SUBNET	Full subnet resource path: projects/neustar-lz/regions/us-central1/subnetworks/...

3.2 Build Environment Variables

BUILD_UUID	Unique short UUID generated per build run via uuidgen
WORKSPACE	Jenkins/shell working directory — set to \$PWD
TMPDIR	Packer temp directory: \${WORKSPACE}/packer_cache/tmpdir
PACKER_SSH_KEYS	Directory holding generated SSH keypair for Packer
LIBGUESTFS_PATH	Path to libguestfs appliance (legacy VMware builds)
ANSIBLE_VAULT_PASSWORD_FILE	Jenkins credential binding for Ansible Vault

3.3 Credential Management

All secrets are managed through Jenkins credentials and never stored in the repository. The `.gca` file (GCP service account JSON) is symlinked from the Jenkins-injected credential path at the start of each pipeline stage. The `.gitignore` explicitly excludes `.gca`, `ssh-keys/`, `packer_cache/`, and other generated artifacts.

Note: The `.env` file in the repository contains non-secret infrastructure references (hostnames, datacenter names). All credential values use variable references filled at runtime.

4. Packer Configuration

Packer orchestrates the image build. Four HCL files define the complete build pipeline. Required plugins are declared in `gcp-builds.pkr.hcl` and automatically downloaded by **packer init** before the build runs.

4.1 `variables.pkr.hcl` — Input Variables

All configurable parameters are declared here. Defaults pull from environment variables using the `env()` function:

Variable	Type	Default	Purpose
<code>build_number</code>	<code>string</code>	<code>"0"</code>	Jenkins BUILD_NUMBER passed at runtime
<code>gcp_project_id</code>	<code>string</code>	<code>env(GCP_PROJECT_ID)</code>	GCP project to build in
<code>gcp_credentials_file</code>	<code>string</code>	<code>env(GCP_CREDENTIALS_FILE)</code>	Path to service account JSON
<code>gcp_zone</code>	<code>string</code>	<code>"us-central1-a"</code>	GCP zone for the build VM
<code>gcs_bucket</code>	<code>string</code>	<code>env(GCS_BUCKET)</code>	GCS bucket for RAW image export
<code>ssh_public_key</code>	<code>string</code>	<code>env(PACKER_SSH_PUBLIC_KEY)</code>	SSH key injected into instance metadata
<code>post</code>	<code>list(object)</code>	<code>[]</code>	Dynamic list of builds to run (name, target, groups)

4.2 `gcp-sources.pkr.hcl` — Source Block

Defines a **googlecompute** source named `debian12`. This source represents the temporary GCP VM that Packer launches, configures, and then snapshots into a new GCE image:

<code>source_image</code>	debian12-hardened-20260626-22 — the pre-hardened base image
<code>source_image_project_id</code>	odev-eiaas-dev-vmi-archv-b9ba — project hosting the base image
<code>machine_type</code>	n2-standard-4 — 4 vCPU, 16 GB RAM for the build VM
<code>disk_size / disk_type</code>	75 GB SSD (pd-ssd)
<code>network / subnetwork</code>	Internal VPC: neustar-vpc-lz1 / neustar-shared-infra-usc1-dev-subnet-51d7
<code>ssh_username / ssh_pty</code>	User: debian, PTY enabled, 3-hour SSH timeout
<code>omit_external_ip</code>	TRUE — build VM has no public IP; reaches Artifactory via internal network
<code>image_name</code>	debian12-hardened-YYYYMMDD-{build_number} (formatted at build time)
<code>image_family</code>	debian12-hardened — allows latest-image lookups by family name
<code>metadata ssh-keys</code>	Packer SSH public key injected; OS Login disabled (enable-oslogin=FALSE)
<code>tags</code>	["packer-build"] — enables firewall rules for SSH during build

4.3 `gcp-builds.pkr.hcl` — Build Block

The build block wires together the source, Ansible provisioner, and two post-processors. A **dynamic source** block iterates over the `var.post` list, enabling multiple image variants to be built in a single Packer run:

#	Stage	Description
1	Plugin Declaration	Requires Packer >= 1.11.0, googlecompute >= 1.1.0, ansible >= 1.1.2
2	Dynamic Source	Iterates var.post list; creates one build per entry with name@target labeling
3	Ansible Provisioner	Runs ansible/site.yml as user debian with --become root; installs Galaxy roles/collections to per-build packer_cache dirs
4	Post-processor 1: OpenSCAP	Runs bin/openscap.sh to score the scan results XML; fails build if score is below threshold
5	Post-processor 2: GCS Export	Runs bin/export-from-gcp.sh to export the GCE image as RAW tar.gz into the GCS bucket under post/{source.name}.tar.gz

4.4 os-debian.pkvars.hcl — Debian Variable Overrides

Provides the *post* variable value that drives the dynamic source loop. Currently defines one build variant:

```
post = [{ name="debian12", target="local_artifact",  
groups=["debian12","debian","local_artifact"]} ]
```

The **name** becomes the Ansible host group, **target** describes the artifact destination, and **groups** are the Ansible inventory groups applied to this build.

5. Ansible Automation

Ansible performs all in-guest configuration of the Packer build VM. The playbook tree follows a layered include pattern: a single entry-point delegates to OS-specific playbooks which import reusable task files.

5.1 Playbook Hierarchy

#	Stage	Description
1	ansible/site.yml	Packer entry point (referenced by provisioner <code>playbook_file</code>)
2	ansible/playbooks/build.yml	Universal entry point — imports <code>gcp_build.yml</code>
3	ansible/playbooks/gcp_build.yml	GCP Debian 12 build — imports three task files in order
4	tasks/setup_apt_repos.yml	Step 1: Update apt cache, configure internal Pulp mirror repos
5	tasks/setup_gcp_boot.yml	Step 2: Install GCP agents, configure GRUB for serial console
6	tasks/clean_apt.yml	Step 3: Autoremove packages, purge apt cache, remove apt lists

5.2 Task Files — Detailed

setup_apt_repos.yml

Uses the **ansible.builtin.deb822_repository** module to write modern `.sources` format APT repository files. Two repos are configured (from `group_vars/debian12.yml`):

Repo Name	Purpose	URI	Suite / Component
tu_debian12_main	Main Debian 12 packages	https://pulp-ch.transunion.com/pulp/content/debian12/main	bookworm / main
tu_debian12_security	Security updates	https://pulp-ch.transunion.com/pulp/content/debian12/security	bookworm-security / main

setup_gcp_boot.yml

Prepares the image to boot correctly on GCP:

- Installs **google-guest-agent** and **google-osconfig-agent** via apt
- Enables both services via systemd (`systemctl enable`)
- Patches **/etc/default/grub** `GRUB_CMDLINE_LINUX` for serial console: `console=ttyS0,38400n8d net.ifnames=0 biosdevname=0`
- Runs **update-grub** only when the GRUB file was actually changed (idempotent)

clean_apt.yml

Minimizes image size before snapshot:

- apt autoremove --purge to remove orphaned packages
- apt clean to purge `/var/cache/apt/archives`
- Removes `/var/lib/apt/lists` (package index cache)

- Recreates the empty /var/lib/apt/lists directory with correct permissions (root:root 0755)

5.3 Inventory & Group Variables

group_vars/all/main.yml	tu_content_server: chmgl9ap27.corp.transunion.com tu_pulp_server: pulp-ch.transunion.com
group_vars/debian12.yml	setup_apt_repo_params list — drives setup_apt_repos.yml task loop

***Note:** The Packer Ansible provisioner assigns groups dynamically from the post variable. This means group_vars/debian12.yml is automatically applied to all debian12 builds.*

5.4 Ansible Galaxy Requirements

Three Ansible collections are installed before each build:

ansible.posix >= 1.5.0	POSIX-specific modules (sysctl, mount, authorized_key, etc.)
community.general >= 8.0.0	Large community collection (deb822_repository, etc.)
google.cloud >= 1.3.0	GCP modules used by Molecule (gcp_compute_instance)

6. Jenkins CI/CD Pipeline

The Jenkinsfile defines a declarative pipeline with a 120-minute global timeout. The pipeline runs on a dedicated agent labeled **ifa-packer-build** and injects all secrets via Jenkins credential bindings in the environment block.

6.1 Pipeline Stages

#	Stage	Description
1	Initialize	Runs bin/init-env.sh: checks OS version, creates Python venv, installs requirements.txt, prints tool versions
2	Validation	Checks out ansible_project_cloud_infra repo; symlinks GCP keyfile; runs packer validate (dry-run) and ansible-lint + yamllint + flake8
3	Build	Symlinks GCP keyfile; clears old manifest; runs full packer build; archives OpenSCAP HTML reports and packer-manifest.json as Jenkins artifacts
4	Deploy	Conditional on branch=story-gcp; runs bin/deploy_gcp.sh to push built images to GCP via gcloud
5	Post (always)	cleanWs() removes workspace except .venv and .git directories

6.2 Jenkins Credentials Used

ansible_vault_pwd_file	File credential — Ansible Vault password file
ifa-gca	Secret file — GCP service account JSON keyfile
gcs-packer-bucket	Secret text — GCS bucket name for image export
848c1077-5601-48a8-b65d-39b97f78642b	SSH credential — Git clone of ansible_project_cloud_infra

6.3 Branch Strategy (docs/pipeline-steps.yml)

main	On merge: sync AWX, deploy artifacts, tag branch, send notifications
story-*	On push: ansible-lint, python-lint, local-builds
release-x.y.z	On merge: sync AWX to release, run remote-builds, remote-test, archive artifacts
pipeline-*	On push: same as story — lint and local builds

7. Molecule Testing Framework

Molecule provides integration testing for the Ansible roles/playbooks. The test suite in `ansible/extensions/molecule/default/` spins up real GCP instances using the `google.cloud` collection and runs the full build playbook against them.

7.1 Test Lifecycle

#	Stage	Description
1	destroy (pre)	Delete any lingering GCP instances from a previous test run
2	list	List Molecule-managed instances (verification step)
3	create	Spin up GCP n2-standard-2 instances from the source image via <code>google.cloud.gcp_compute_instance</code>
4	verify	Run <code>ansible/site.yml</code> against the live instances (converge + verify)
5	destroy (post)	Delete all created GCP instances regardless of test outcome

7.2 Molecule Platform Configuration

Platform name	debian12-gcp-01
Source image	debian12-hardened-20260626-22 from odev-eiaas-dev-vmi-archv-b9ba
Machine type	n2-standard-2 (2 vCPU, 8 GB RAM for test instances)
Disk	75 GB pd-ssd
Network	Same VPC as Packer: neustar-vpc-lz1 / neustar-shared-infra-usc1-dev-subnet-51d7
SSH user	debian — same as Packer build
Ansible groups	["debian12", "debian", "local_artifact"]
group_vars link	Symlinked to <code>../../inventories/group_vars</code> so same variables apply

7.3 JUnit Integration (CI)

When running inside Jenkins (`JENKINS_HOME` is set) and `JUNIT_DISABLED` is not set, the verify step uses the Ansible JUnit callback plugin. This produces XML test results that Jenkins parses for the test results trend graph. Key JUnit settings:

<code>ANSIBLE_STDOUT_CALLBACK</code>	junit — enables JUnit XML output
<code>JUNIT_FAIL_ON_IGNORE</code>	true — ignored tasks count as failures
<code>JUNIT_FAIL_ON_CHANGE</code>	true — changed tasks count as failures (idempotency check)
<code>JUNIT_OUTPUT_DIR</code>	<code>\${WORKSPACE}/junit</code>

8. Python Task Runner (tasks.py)

tasks.py uses the **PyInvoke** library to define runnable tasks that abstract build steps behind consistent commands. All tasks are called via **bin/invoke.sh** which activates the Python venv first.

Task	Description
execute_build	Master build task: clean, make folders, generate SSH keypair, packer init, packer validate, packer build. Supports --dry-run and --skip-prep flags
generate_key_pair	Creates RSA 4096-bit SSH keypair in ssh-keys/ (skips if already exists)
static_code_check	Runs ansible-lint, yamllint, and flake8 in parallel using async promises
clean_all	Removes ssh-keys/, artifacts.pkvars.json, packer_cache/, and Ansible inventories
clean_builds	Removes builds/ and packer_cache/, recreates folder structure
make_folders	Creates packer_cache/, packer_cache/tmpdir/, ssh-keys/
validate_branch_name	Validates git branch matches allowed prefixes: main, story-, release- (with semver suffix)
fetch_images	Downloads source images from Artifactory using jfrog rt download
molecule_run_test	Orchestrates full Molecule test lifecycle: destroy→list→create→verify→destroy with JUnit support
molecule_get_build	Parses packer-manifest.json to extract the build timestamp for .env.yml
molecule_set_env.yml	Writes IMG_BLD_TIME to .env.yml and symlinks it into the molecule directory
purge_outdated_gcp_images	Keeps last 5 images per family (debian12-hardened), deletes older ones via gcloud

9. Shell Scripts (bin/)

The `bin/` directory contains shell scripts that wrap common operations. They are designed to be called by Jenkins stages or by `invoke` tasks.

`init-env.sh` — Environment bootstrap

- Reads `/etc/os-release` to detect RHEL/CentOS version (8.9, 8.10, 7.9)
- Selects correct Python binary (`python3.9` or `rh-python38`)
- Creates `.venv`, activates it, installs `requirements.txt`
- Prints tool versions: `invoke`, `yamllint`, `ansible-lint`, `flake8`

`invoke.sh` — PyInvoke shim

- Sources `.src` to load `venv` environment variables
- Passes all arguments through to `invoke`
- Used as the canonical way to call any `invoke` task

`deploy_gcp.sh` — GCP image deployment

- Sources `.env` for `GCP_PROJECT`, `GCP_KEYFILE`, `GCP_SUBNET`
- Authenticates to GCP via `gcloud auth activate-service-account`
- Reads `packer-manifest.json` to get built VMDK file paths
- Runs `gcloud compute images import` in parallel for each build
- Waits for all background processes; exits non-zero on any failure

`deploy-lab.sh` — Nutanix lab deployment (legacy)

- Uploads `qcow2` image to Artifactory (`storage-local/ifa/nutanix`) with retry logic
- Calls `nux-upload-img.py` to register the image in Nutanix Prism Central
- Used for pre-integration testing before production pipeline

10. End-to-End Workflow

The following describes the complete flow from a code push to a published GCP image, as executed by the Jenkins pipeline.

#	Stage	Description
1	Developer pushes code	Code pushed to feature/debian12-gcp branch; Jenkins pipeline triggered automatically
2	Stage: Initialize	Jenkins agent runs bin/init-env.sh — Python venv created, all tools installed and version-verified
3	Stage: Validation — Checkout	ansible_project_cloud_infra cloned from Bitbucket into workspace
4	Stage: Validation — Packer Validate	GCP keyfile symlinked; invoke.sh execute-build --dry-run runs packer init + packer validate without building
5	Stage: Validation — Static Code Check	ansible-lint, yamllint, flake8 run in parallel; any failure aborts the pipeline
6	Stage: Build — Pre-build Cleanup	packer_cache/packer-manifest.json removed to ensure fresh manifest
7	Stage: Build — Packer Execution	invoke.sh execute-build --skip-prep calls packer build with os-debian.pkvars.hcl overrides
8	Packer: Source VM Launch	Packer launches n2-standard-4 GCP VM from debian12-hardened source image in us-central1-a
9	Packer: SSH Connection	Packer waits for SSH (user: debian) with up to 3-hour timeout; no public IP — internal network only
10	Packer: Ansible Provisioning	Ansible runs ansible/site.yml with --become root; Galaxy collections installed to packer_cache/ansible/
11	Ansible: setup_apt_repos	Internal Pulp mirror repos configured for Debian 12 main and security packages
12	Ansible: setup_gcp_boot	google-guest-agent installed and enabled; GRUB updated for GCP serial console
13	Ansible: clean_apt	APT cache purged, lists removed — image size minimized
14	Packer: Image Snapshot	Packer stops the VM and creates GCE image: debian12-hardened-YYYYMMDD-{build_number}
15	Post-processor: OpenSCAP	bin/openscap.sh validates CIS compliance score against scan results XML
16	Post-processor: GCS Export	bin/export-from-gcp.sh exports image as RAW tar.gz to GCS bucket under post/debian12.tar.gz
17	Jenkins: Archive Artifacts	OpenSCAP HTML report and packer-manifest.json archived in Jenkins
18	Stage: Deploy (story-gcp branch)	bin/deploy_gcp.sh authenticates to GCP and imports built images
19	Post-pipeline: Cleanup	cleanWs() removes workspace (preserving .venv and .git)

20	Optional: Molecule Testing	tasks.py molecule_run_test can be triggered separately to spin up live GCP test instances and verify the image
----	-----------------------------------	--

11. Key Design Decisions

No Public IP on Build VM

`omit_external_ip=true` and `use_internal_ip=true` ensure the Packer build VM never gets a public IP. Access to Artifactory/Pulp mirrors goes through the internal VPC network, reducing attack surface.

Internal Package Mirror (Pulp)

All APT packages are pulled from the internal Pulp mirror (`pulp-ch.transunion.com`) rather than the public Debian mirrors. This ensures reproducibility, air-gap capability, and compliance with internal change control.

Dynamic Packer Source Blocks

The dynamic source block in `gcp-builds.pkr.hcl` iterates `var.post`, allowing multiple image variants (e.g., `debian12@local_artifact`) to be built in parallel in a single packer build invocation without duplicating HCL.

Separate Source and Build HCL Files

Splitting source definitions (`gcp-sources.pkr.hcl`) from build logic (`gcp-builds.pkr.hcl`) follows the Packer recommended pattern and makes it easy to add new source types (e.g., VMware) without modifying the build block.

Per-Build Ansible Role Cache

Galaxy roles and collections are installed to `packer_cache/ansible/roles/{source.name}` and `collections/{source.name}`. This isolates dependencies between parallel builds and avoids cross-contamination.

RAW Export to GCS

Images are exported as RAW format (`disk.raw` inside `tar.gz`) to GCS. RAW is GCP's native import/export format with no conversion overhead, and `tar.gz` can be re-imported into any GCP project directly.

Image Retention Policy

The `purge_outdated_gcp_images` task keeps only the last 5 images per family. This prevents unbounded GCS/GCE image storage growth while maintaining enough history to roll back.

Ansible-First Configuration

All guest configuration is done via Ansible (not shell provisioners). This makes the build logic testable via Molecule and reusable across Packer and AWX runtime.

12. File Reference Index

Complete listing of all meaningful project files with their roles:

Cat.	File	Description
PACKER	gcp-builds.pkr.hcl	Main Packer HCL: plugins, locals, build block, Ansible provisioner, post-processors
PACKER	gcp-sources.pkr.hcl	GCP googlecompute source definition: VM spec, networking, image naming
PACKER	variables.pkr.hcl	All Packer input variable declarations with env() defaults
PACKER	os-debian.pkvars.hcl	Debian 12 variable overrides: defines the post build list
PACKER	pkrrvars-dump/packer.pkr.hcl	Reference/example pkrrvars structure
ANSIBLE	ansible/playbooks/build.yml	Universal Ansible entry point — imports gcp_build.yml
ANSIBLE	ansible/playbooks/gcp_build.yml	GCP Debian 12 build playbook — hosts, tasks chain
ANSIBLE	ansible/playbooks/tasks/setup_apt_repos.yml	Configure internal Pulp APT mirrors
ANSIBLE	ansible/playbooks/tasks/setup_gcp_boot.yml	Install GCP agents, fix GRUB for serial console
ANSIBLE	ansible/playbooks/tasks/clean_apt.yml	Purge APT cache and lists to minimize image size
ANSIBLE	ansible/requirements.yml	Ansible Galaxy collection requirements
ANSIBLE	ansible/inventories/group_vars/all/main.yml	Global vars: content server and Pulp server URLs
ANSIBLE	ansible/inventories/group_vars/debian12.yml	Debian 12 APT repo parameters
MOLECULE	ansible/extensions/molecule/default/molecule.yml	Molecule platform: GCP instance spec, driver, verifier
MOLECULE	ansible/extensions/molecule/default/create.yml	Spin up GCP test instances via google.cloud.gcp_compute_instance
MOLECULE	ansible/extensions/molecule/default/converge.yml	Run build playbook on test instances
MOLECULE	ansible/extensions/molecule/default/verify.yml	Post-converge assertions
MOLECULE	ansible/extensions/molecule/default/destroy.yml	Delete GCP test instances
MOLECULE	ansible/extensions/molecule/default/vars/main.yml	Molecule-specific variable definitions
JENKINS	Jenkinsfile	Declarative Jenkins pipeline: 4 stages + post cleanup
JENKINS	docs/pipeline-steps.yml	Branch strategy and action definitions
PYTHON	tasks.py	PyInvoke task definitions: build, test, clean, purge, validate
PYTHON	requirements.txt	Python package dependencies
PYTHON	invoke.yml	Invoke runner configuration (echo format)
SHELL	bin/init-env.sh	Environment bootstrap: venv creation, dependency install
SHELL	bin/invoke.sh	Shim to call invoke tasks with venv activated
SHELL	bin/deploy_gcp.sh	Deploy built images to GCP via gcloud

SHELL	bin/deploy-lab.sh	Upload images to Nutanix lab via Artifactory
CONFIG	.env	Shell environment variable exports (non-secret)
CONFIG	artifactory-images.json	Image catalog: Artifactory paths, groups, variants
CONFIG	.gitignore	Excludes: .gca, .venv, ssh-keys/, packer_cache/, builds/
CONFIG	.ansible-lint	ansible-lint rules: exclude paths, warn list, skip list
CONFIG	.yamllint	yamllint rules: indentation, line length, trailing spaces
CONFIG	VERSION	Project version: 0.0.1

Category Legend:

PACKER	ANSIBLE	MOLECULE	JENKINS	PYTHON	SHELL	CONFIG
--------	---------	----------	---------	--------	-------	--------